

Kingsgate / Semar

Architecture Documentation

Tech stack, data flow, database schemas, backend & frontend

Index

1. Tech Stack
 2. Data Flow & Communication (FE ↔ BE)
 3. Third-Party Services
 4. Database Schemas – Table List
 5. Foreign Key Relationships (by domain)
 6. Backend Architecture
 7. Frontend (Dashboard) Architecture
 8. Payment Page Details
 9. Payin Order Architecture
-

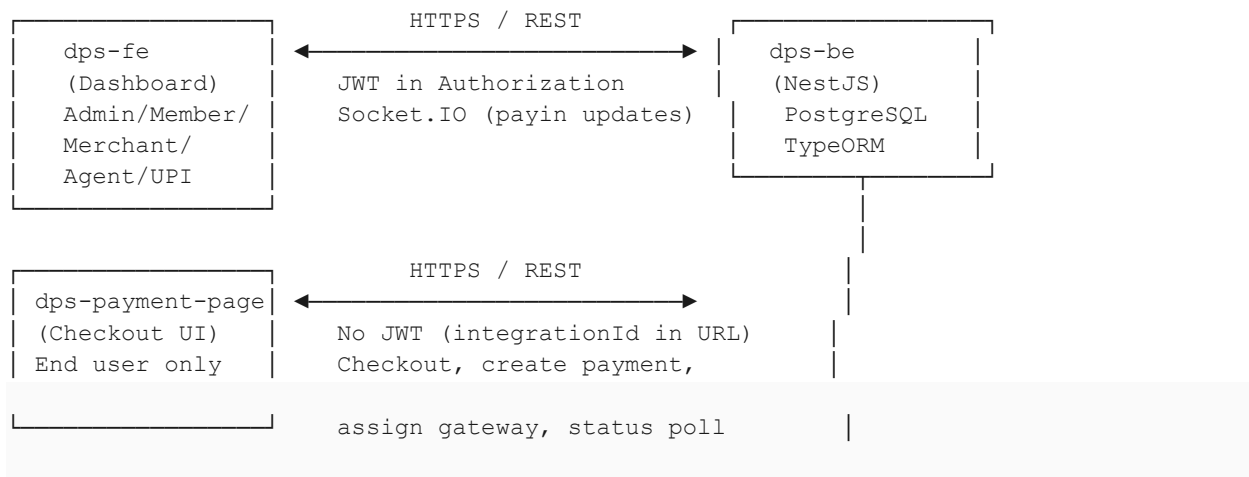
1. Tech Stack

Layer	Technology	Notes
Backend (dps-be)	NestJS 10, Node.js, TypeScript	REST API, WebSockets (Socket.IO)
Database	PostgreSQL	TypeORM, snake_case naming
Frontend Dashboard (dps-fe)	React 18, TypeScript, Vite	SPA

UI (Dashboard)	Mantine 8, React Router, Redux Toolkit, Axios	Charts (Mantine Charts, Recharts), Socket.IO client
Payment Page (dps-payment-page)	React 18, TypeScript, Vite	Standalone SPA
UI (Payment Page)	Mantine 7, React Router, Axios	No auth; checkout flows only
Email	SendGrid	Transactional emails (e.g. OTP, notifications)
File Storage	AWS S3	Receipts, images (uploaded via BE)
Payment Gateways (3rd PG)	Razorpay, PhonePe, PayU, Cashfree, UniqPay	Integrated in BE; used for payin/payout

2. Data Flow & Communication (FE ↔ BE)

2.1 High-Level Flow



2.2 Dashboard (dps-fe) ↔ Backend

- **Auth:** Login (identity) → JWT returned → stored (e.g. localStorage); subsequent requests send `Authorization: Bearer <token>`.
- **API base URL:** Configured in env (e.g. `VITE_API_BASE_URL`). All API modules in `src/api/` use Axios with base URL and attach token via interceptor.
- **Real-time:** Socket.IO client connects to BE for payin order updates (status, UTR, etc.); used on merchant/member/admin order screens.
- **Flow:** User action in UI → API call (GET/POST/PUT/PATCH/DELETE) → BE controller → service → TypeORM → PostgreSQL → response → UI update.

2.3 Payment Page (dps-payment-page) ↔ Backend

- **No login:** Payment page is public; identifies context via `integrationId` (checkout) or `orderId` (member/UPI vendor flow).
- **API base URL:** `VITE_API_BASE_URL` points to same dps-be.
- **Key flows:**
 - **Checkout:** POST `/payment-system/checkout/:integrationId` → get channels, amount, business name.
 - **Create order:** POST `/payment-system/create-payment-order` → get `systemOrderId`.
 - **Assign gateway:** POST `/payment-system/assign-payment-gateway` → get redirect URL or member/UPI vendor details.
 - **Member / UPI vendor channel:** GET `member-channel` or `upi-vendor-channel`, submit UTR/receipt, poll status.

3. Third-Party Services

3.1 SendGrid (Email)

- **Purpose:** Sending transactional emails (e.g. OTP, password reset, notifications).
- **Integration:** Used in dps-be via `@sendgrid/mail` in `src/services/email/email.service.ts`.
- **Config:** `SENDGRID_API_KEY` in environment. From address configurable in code.
- **Usage:** `EmailService.send({ subject, text, receiver, attachment? });` optional PDF attachment (e.g. for receipts).

3.2 AWS S3 (File Storage)

- **Purpose:** Storing uploaded files – payment receipts (images), other images used by the platform.
- **Integration:** dps-be uses `@aws-sdk/client-s3` in `src/upload/upload.service.ts`.
- **Config:** `AWS_REGION`, `AWS_ACCESS_KEY`, `AWS_SECRET_KEY`, `S3_BUCKET_NAME`.
- **Flow:** Client uploads file to BE (e.g. `POST /upload/receipt` with multipart); BE generates key (e.g. `payinOrderId_uniqid`), uploads to S3, returns public URL.
- **Receipts:** Stored and linked to payin orders (e.g. `transactionReceipt` field stores URL). PDF generation may reference S3 URLs for images.

4. Database Schemas – Table List

All tables use **snake_case** naming (TypeORM `SnakeNamingStrategy`). Below: logical table name and main columns (PK/FK and key fields).

Table (Entity)	Key Columns (summary)
identity	id, email, password, userType (enum), beneficiaryId (Cashfree)
ip	id, value, identity (FK)
admin	id, identity_id (FK), firstName, lastName, role (SUPER_ADMIN/SUB_ADMIN), permission* flags

merchant	id, identity (FK), agent (FK), integrationId, businessUrl, balance, payinChannels, payoutChannels, payinServiceRate, payoutServiceRate, withdrawalServiceRate, min/max Payout/Withdrawal, allowMemberChannelsPayin/Payout, enableUpiVendorGateway, organizationId, apiKey
member	id, identity (FK), agent (FK, self-ref), teamId, quota, balance, firstName, lastName, singlePayoutUpperLimit, dailyTotalPayoutLimit, agentCommissions (JSON)
agent	id, identity_id (FK), agent_id (FK, self-ref), organizationId, balance, withdrawalPassword, withdrawalRate, min/maxWithdrawalAmount, agentCommissions (JSON)
upi_vendor	id, identity (FK), firstName, lastName, enabled, commissionRate, settlementUpId
sub_merchant	id, merchant_id (FK), identity_id (FK), firstName, lastName, permissionSubmitPayouts/Withdrawals
end_user	id, merchant_id (FK), userId, name, email, mobile, upiDetails/netBankingDetails/eWalletDetails (JSON), totalPayinAmount, totalPayoutAmount, beneficiaryId, fundAccountId
payin	id, systemOrderId, merchantOrderId, enduser_id (FK), merchant (FK), member (FK), upiVendor (FK), amount, status, channel, callbackStatus, payinMadeOn, gatewayName, gatewayPaymentLink, transactionId, transactionReceipt, successUrl, failureUrl

payout	id, systemOrderId, merchantOrderId, user (FK EndUser), merchant (FK), member (FK), amount, status, channel, notificationStatus, payoutMadeVia, gatewayName, utr, transactionReceipt
withdrawal	id, systemOrderId, identity_id (FK), amount, status, channel, channelDetails, withdrawalMadeOn, gatewayName, transactionId, transactionReceipt
topup	id, systemOrderId, member (FK), amount, status, channel, notificationStatus, transactionId, transactionReceipt
settlement	id, systemOrderId, upi_vendor (FK), upi (FK), settlementAmount, paidAmount, remainingAmount, status, transactionId
transaction_update	id, identity_id (FK), payin_order_id (FK), payout_order_id (FK), withdrawal_order_id (FK), topup_order (FK), orderType, userType, systemOrderId, amount, before, after, pending
fund_record	id, identity_id (FK), orderType, systemOrderId, merchantOrderId, balanceType, amount, serviceFee, description, before, after, type (TRANSACTION/FROZEN)
agent_referral	id, agent_id (FK), referred_agent_id (FK), referred_merchant_id (FK), referralCode, agentType (merchant/agent), status, payinCommission, payoutCommission, merchantPayinServiceRate (JSON)

member_referral	id, member_id (FK), referred_member_id (FK), referralCode, payinCommission, payoutCommission, topupCommission, status
organization	organizationId (UUID), leader (FK Agent), organizationSize, totalReferralCommission
team	teamId (UUID), team_leader (FK Member), teamSize, totalReferralCommission, totalPayinCommission, totalPayoutCommission, totalQuota, team*CommissionRate
upi	id, identity (FK), upiId, mobile, isBusinessUpi, channelIndex, beneficiaryName, enabled, title, isUpiVendor, settlementAmount
net_banking	id, identity (FK), accountNumber, ifsc, bankName, beneficiaryName, channelIndex
e_wallet	id, identity (FK), app, mobile, channelIndex, beneficiaryName
channel_settings	id, gatewayName, type, channelName, enabled, minAmount, maxAmount, upstreamFee
razorpay	id, incoming, outgoing, key_id, key_secret, account_number, sandbox_*
phonepe, payu, cashfree, uniqlpay	Gateway config tables (credentials, sandbox/live)

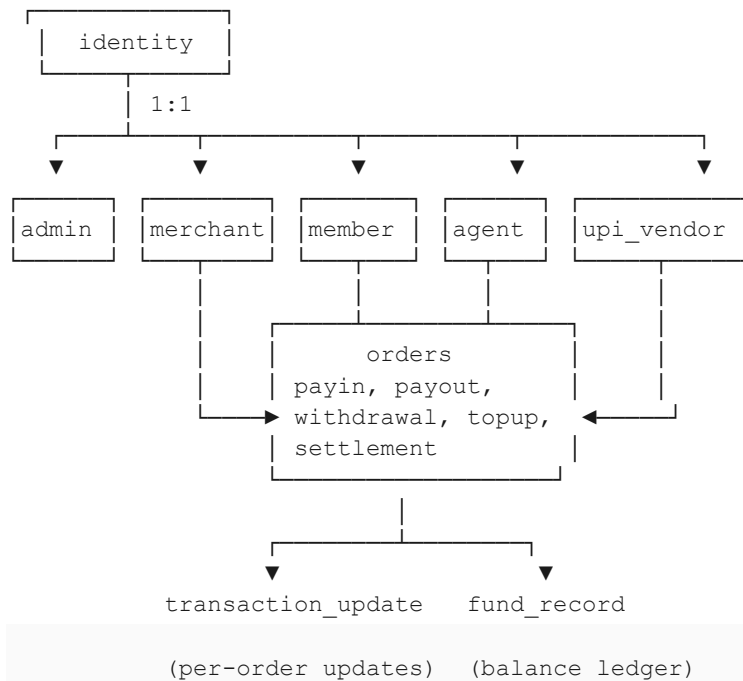
payin_mode	id, merchant (FK), type (DEFAULT/PROPORTIONAL/AMOUNT RANGE), number
proportional_payin_mode, amount_range_payin_mode	FK to payin_mode; range config for merchant payin mode
system_config	id, payinTimeout, payoutTimeout, currency, defaultPayinGateway/defaultWithdrawalGateway/defaultPayoutGateway (JSON), payinSystemProfitRate, topupThreshold/topupAmount/topupServiceRate, *CommissionRateForMember, *ForMerchant, withdrawalRate, min/maxWithdrawalAmount, systemProfit, frozenAmountThreshold
notification	id, for (user id), type, status, data (JSON)
alert	id, for, type, userType, status, data (JSON)

5. Foreign Key Relationships (by Domain)

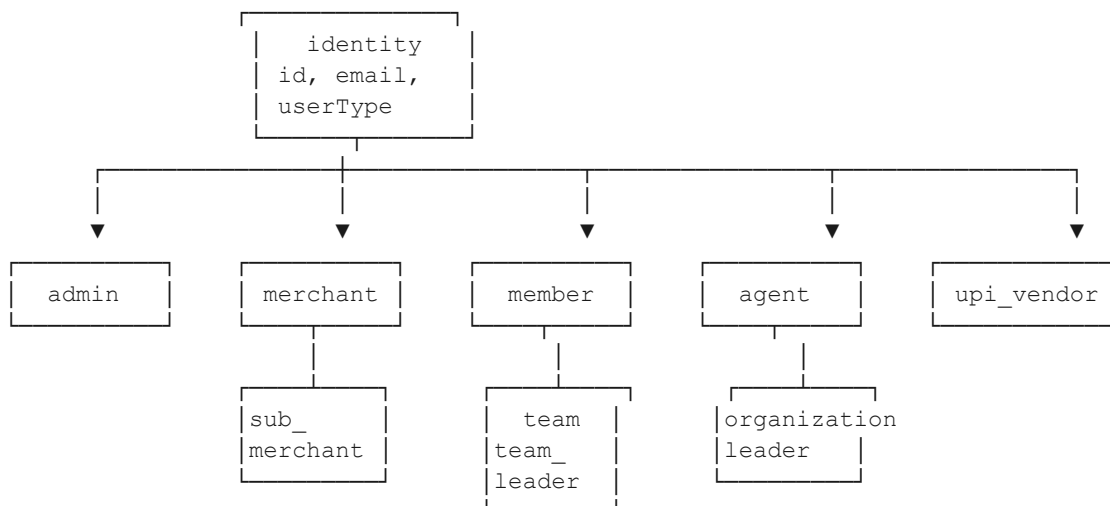
Relationships are grouped by **similarity or operation**. An entity may appear in more than one diagram.

5.1 High-Level Database Diagram (Core Entities Only)

Simplified view: main user types and order tables. Arrows mean “references” (FK).



5.2 Identity & User Entities



```
identity 1—* ip
```

```
identity 1—* net_banking, e_wallet, upi (channel details per identity)
```

5.3 Order Entities (Payin, Payout, Withdrawal, Topup, Settlement)

```
merchant —* payin      member —* payin      upi_vendor —* payin      end_user —* payin
merchant —* payout     member —* payout     end_user —* payout
identity —* withdrawal
member —* topup
upi_vendor —* settlement  upi —* settlement
```

```
payin —* transaction_update  payout —* transaction_update
withdrawal —* transaction_update  topup —* transaction_update
```

```
identity —* fund_record (links balance changes to user)
```

5.4 Referrals & Organisation

```
agent_referral: agent —* agent_referral (agent who created code)
                  referred_agent_id → agent (optional: referred agent)
                  referred_merchant_id → merchant (optional: referred
merchant)
```

```
member_referral: member —* member_referral (member who created code)
                  referred_member_id → member (referred member)
```

```
merchant.agent_id → agent (merchant's referring agent)
member.agent → member (member's referring member agent)
member.teamId / team.team_leader → team / member (team leader)
```

```
agent.organization / organization.leader → organization / agent
```

5.5 Channels & Gateway Config

```
channel_settings: gatewayName, channelName, type, minAmount, maxAmount, upstreamFee
(standalone config)
```

```
razorpay, phonepe, payu, cashfree, uniqlpay: gateway credentials (one row per
gateway type; no FK to users/orders)
payin_mode → merchant (1:1)
```

```
proportional_payin_mode, amount_range_payin_mode → payin_mode
```

5.6 Low-Level FK Reference

From Table	FK Column	To Table
ip	identity	identity
admin	identity_id	identity
merchant	identity	identity
merchant	agent	agent
member	identity	identity
member	agent	member
member	teamId → team_leader	team
agent	identity_id	identity
agent	agent_id	agent
upi_vendor	identity	identity

sub_merchant	merchant_id	merchant
sub_merchant	identity_id	identity
end_user	merchant	merchant
payin	enduser_id, merchant, member, upiVendor	end_user, merchant, member, upi_vendor
payout	user, merchant, member	end_user, merchant, member
withdrawal	identity_id	identity
topup	member	member
settlement	upiVendor, upi	upi_vendor, upi
transaction_up date	identity_id, payin_order_id, payout_order_id, withdrawal_order_id, topup_order	identity, payin, payout, withdrawal, topup
fund_record	identity_id	identity

agent_referral	agent_id, referred_agent_id, referred_merchant_id	agent, agent, merchant
member_referral	member_id, referred_member_id	member, member
organization	leader	agent
team	team_leader	member
upi, net_banking, e_wallet	identity	identity
payin_mode	merchant	merchant

6. Backend Architecture

6.1 Structure (dps-be)

NestJS modular structure under `src/`. Each domain has a **module**, **controller**, **service(s)**, and optionally **entities**, **DTOs**.

```
src/
├─ app.module.ts           # Imports all feature modules, TypeORM, Config
├─ main.ts                 # Bootstrap (NestFactory, port, CORS, etc.)
├─ identity/               # Login, JWT, user type resolution
├─ admin/, merchant/, member/, agent/, sub-merchant/, upi-vendor/
├─ payin/, payout/, withdrawal/, topup/, settlement/
├─ payment-system/         # Checkout, create order, assign gateway, member/UPI
├─ vendor channel
├─ gateway/               # Gateway CRUD (Razorpay, PhonePe, etc.)
```

```

├─ channel/                  # Channel config (UPI, NetBanking, EWallet entities)
├─ fund-record/, transaction-updates/
├─ agent-referral/, member-referral/
├─ organization/, team/
├─ system-config/
├─ overview/                # Analytics/overview APIs (admin & user)
├─ notification/, alert/, bulletin/
├─ upload/                  # S3 upload (receipts, images)
├─ receipt/                 # Receipt generation/upload handling
├─ end-user/
├─ user-details/            # Aggregated user profile APIs per role
├─ export/                  # Export (e.g. Excel)
├─ socket/                  # Socket.IO gateways (payin updates)
├─ services/                # Shared: email (SendGrid), jwt, cron, pdf-maker
└─ utils/                   # Guards, decorators, interceptors, exception filters

```

6.2 Key Flows

- **Auth:** IdentityController accepts login (email/password, userType); IdentityService validates, returns JWT. Guards on protected routes validate JWT and attach user/role.
- **Payment system:** PaymentSystemController exposes checkout, create-payment-order, assign-payment-gateway, member-channel, upi-vendor-channel, submit-payment, status, order-details. PaymentSystemService orchestrates order creation, gateway assignment, and member/UPI vendor channel logic.
- **Orders:** Payin/Payout/Withdrawal/Topup/Settlement each have admin and role-specific services (e.g. payin-merchant.service, payin-member.service). Controllers paginate and filter by role.
- **Upload:** UploadController receives multipart file; UploadService uploads to S3 and returns URL. Receipt module may link URL to payin.

7. Frontend (Dashboard) Architecture

7.1 Structure (dps-fe)

```

src/
├─ main.tsx, App.tsx
├─ routes/                  # createBrowserRouter: /, /sign-in, /admin/*, /member/*,
/merchant/*, /agent/*, /upi-vendor/*
├─ pages/
│   └─ SignIn/, SignUp/, ForgotPassword/
│   └─ Dashboard/          # Layout + sidebar; dashboardRoutes (admin, member,
merchant, agent, upiVendor)

```

```
└─ ThemeProvider/, config/, types/, utils/
```

7.2 Data Flow

- User logs in → token stored; router redirects by role to `/admin`, `/merchant`, etc.
- Each dashboard section (e.g. Payin Orders) calls API from `api/` (e.g. `order.ts`); response is set in local state or used in table; pagination/filters passed as query params.
- Socket connection (e.g. `useSocket`) subscribes to room by user/order for real-time payin updates.

8. Payment Page Details

8.1 Purpose

Standalone React app for **end users** to complete payment. No login; access via checkout link (integrationId) or order link (orderId).

8.2 Structure (dps-payment-page)

```
src/
├─ App.tsx, main.tsx
├─ routes/                # /checkout/:integrationId, /gateway-callback,
                           /payment/:orderId, /upi-vendor-gateway/:orderId
├─ components/
│   ├─ CheckoutPage/      # PaymentAmount, PaymentMethodsList, PaymentMethodCard,
                           PaymentStatusDisplay, LoadingState, ErrorState
│   ├─ GatewayPage/       # Redirect to gateway; general gateway callback handler
│   ├─ MemberChannelPage/ # UPI, Netbanking, EWallet instructions + receipt upload
│   ├─ VendorGatewayUPIPage/
│   └─ TestPage/
├─ hooks/                 # useCheckout, usePaymentProcessing, usePaymentStatus,
                           usePaymentStatusPolling, useWindowMessage
├─ services/              # api.ts (checkout, createPayment, assignGateway,
                           memberChannel, upiVendorChannel, status, orderDetails, uploadReceipt), apiUtils
└─ types/                 # payment.ts (request/response types)
```

```
└─ utils/                                # constants.ts (API_ENDPOINTS, CHANNEL_MAP, etc.),
intent.ts
```

8.3 API Endpoints Used

Endpoint	Purpose
POST /payment-system/checkout/:integrationId	Get checkout config (channels, amount, business name)
POST /payment-system/create-payment-order	Create payin order; returns systemOrderId
POST /payment-system/assign-payment-gateway	Assign gateway; returns redirect URL or member/UPI vendor details
GET/POST member-channel, upi-vendor-channel	Get payment instructions; submit UTR/receipt
GET /payment-system/status	Poll payment status
GET /payment-system/order-details	Order details for status display
POST /upload/receipt	Upload receipt image (S3 via BE)

9. Payin Order Architecture

This section explains how a **payin** flows from “generate payment page” to “success” for all three channel types: **3rd Party Gateway**, **UPI Vendor**, and **Member Channel**. The same payment page (dps-payment-page) is used for all; the backend decides the channel and redirects the user accordingly.

9.1 Getting the Payment Page URL

The merchant (or their server) obtains a URL where the end user can pay. The backend returns this URL when a payin order is created.

Step	Who	Action	Result
1	Merchant / Server	Call POST <code>/payment-system/create-payment-order</code> with <code>integrationId</code> , <code>orderId</code> , <code>amount</code> , <code>userId</code> , <code>channel</code> , <code>environment</code> (and optionally <code>paymentMethod</code> in sandbox).	BE creates a payin order (status INITIATED) and returns <code>{ data: { url: paymentPageUrl } }</code> .
2	Merchant / Server	Use the returned <code>url</code> (or, in SDK redirect flow, follow the redirect from GET <code>create-payment-order-sdk</code>).	Payment page URL looks like: <code>{PAYMENT_PAGE_BASE_URL}/checkout/{integrationId}?orderId={systemOrderId}&apiMode=true&environment=live</code> .

The end user is then sent to this URL (e.g. opened in browser or redirected).

9.2 What Happens on the Payment Page – Assign Gateway & Redirect

When the user lands on the payment page (`/checkout/:integrationId` with `orderId` in the query), the frontend calls the backend to “assign” a payment channel. The backend chooses either a **3rd party gateway**, a **UPI vendor**, or a **member channel**, and responds with a **redirect** to the URL where the user will actually pay.

Step	Component	Action
1	Payment page (FE)	Call POST <code>/payment-system/assign-payment-gateway</code> with <code>integrationId</code> , <code>systemOrderId</code> , <code>environment</code> (and in <code>sandbox</code> , <code>paymentGateway</code>).
2	Backend	Load payin order. If status is already assigned → redirect to stored <code>gatewayPaymentLink</code> . If status is INITIATED → run channel selection (member / UPI vendor / 3rd PG), create payment link or internal URL, update payin to ASSIGNED, store <code>gatewayPaymentLink</code> and <code>trackingId</code> .
3	Backend	Respond with HTTP redirect (302) to the URL where the user should complete payment. The frontend follows this redirect (browser navigates).

The redirect target depends on the assigned channel:

- **3rd party gateway:** URL is the gateway's payment page (e.g. Razorpay / PhonePe / Cashfree payment link).
- **UPI vendor:** URL is our page `/upi-vendor-gateway/{orderId}?environment=...` (our app shows UPI ID and instructions).
- **Member channel:** URL is our page `/payment/{orderId}?environment=...` (our app shows member's UPI / net banking / e-wallet details).

9.3 Flow by Channel Type

9.3.1 3rd Party Gateway (Razorpay, PhonePe, PayU, Cashfree, etc.)

1. User on payment page
 - FE calls `assign-payment-gateway`
 - BE assigns a gateway, creates payment link via gateway API, stores `trackingId` + `gatewayPaymentLink`
 - BE redirects user to GATEWAY's payment URL
2. User pays on gateway's page (card/UPI/net banking on gateway UI)

3. Gateway redirects user back to OUR payment page:
 - URL:
`{PAYMENT_PAGE_BASE_URL}/gateway-callback?orderId={systemOrderId}&environment=...`
 4. Payment page (gateway-callback route):
 - Polls GET `/payment-system/status/:payinOrderId`
 - BE fetches status from gateway API (using trackingId); when gateway reports paid, BE updates payin to COMPLETE
 - Optionally: gateway also sends webhook to BE; BE can update order and next poll returns SUCCESS
5. When status = SUCCESS (or FAILED), payment page shows success/failure and optionally redirects to merchant success/failure URL.

Summary: Assign gateway → redirect to gateway → user pays → gateway redirects to our `/gateway-callback` → polling (and/or webhook) → status SUCCESS/FAILED → show result.

9.3.2 UPI Vendor Channel

1. User on payment page
 - FE calls `assign-payment-gateway`
 - BE assigns UPI vendor (if merchant has `enableUpiVendorGateway`), gets UPI details from queue, stores url = `/upi-vendor-gateway/{orderId}`
 - BE redirects user to our `/upi-vendor-gateway/{orderId}?environment=...`
 2. User on our "UPI vendor gateway" page:
 - FE calls GET `/payment-system/upi-vendor-channel/:payinOrderId` to get UPI ID and instructions (or details are in `redirect/state`)
 - User pays via any UPI app to the shown UPI ID, then submits UTR / proof on our page
 3. FE submits UTR/receipt to BE (e.g. submit UTR endpoint for UPI vendor flow)
 4. Payment page polls GET `/payment-system/status/:payinOrderId`
 - BE returns status from payin order (SUBMITTED → COMPLETE after verification)
- When status = SUCCESS, show success; optionally redirect to merchant URLs

Summary: Assign gateway → redirect to our UPI vendor page → user pays to UPI ID → user submits UTR → polling → SUCCESS when order is marked complete.

9.3.3 Member Channel

1. User on payment page
 - FE calls `assign-payment-gateway`
 - BE assigns a member (round-robin / config), gets member's payment details, stores url = `/payment/{orderId}`
 - BE redirects user to our `/payment/{orderId}?environment=...`
2. User on our "member channel" page:
 - FE calls GET `/payment-system/member-channel/:payinOrderId` to get UPI ID / net banking / e-wallet details and amount
 - User pays offline (e.g. Google Pay / Paytm to member's UPI, or net banking to member's account), then submits UTR and/or receipt image on our page

3. FE uploads receipt (POST /upload/receipt) and submits UTR/transaction ID to BE (e.g. payin submit endpoint)

4. Payment page polls GET /payment-system/status/:payinOrderId

→ BE returns status (SUBMITTED → COMPLETE after admin/member confirmation)

→ When status = SUCCESS, show success; optionally redirect to merchant URLs

Summary: Assign gateway → redirect to our member payment page → user pays offline to member's details → user submits UTR/receipt → polling → SUCCESS when order is confirmed complete.

9.4 One-Page Overview

Channel	Assign gateway returns (redirect to)	User pays	How success is known
3rd Party Gateway	Gateway's payment URL (e.g. Razorpay link)	On gateway's page (card/UPI/net banking)	Gateway redirects to our /gateway-callback → polling (and/or webhook) → BE updates order to COMPLETE
UPI Vendor	Our page: /upi-vendor-gateway/:orderId	To UPI ID shown on our page; then submit UTR on our page	Polling GET /status → SUCCESS when order marked complete
Member Channel	Our page: /payment/:orderId	Offline to member's UPI/bank/wallet; then submit UTR/receipt on our page	Polling GET /status → SUCCESS when order confirmed complete